

Tư Duy Mới: Developer Là Kiến Trúc Sư & Nhạc Trưởng

Khi AI đảm nhận việc viết code, giá trị của developer không còn nằm ở *tốc độ gõ phím* — mà ở khả năng **thiết kế giải pháp**, **chỉ huy AI**, và **kiểm chứng kết quả**.



Chương 2 Dạy Gì?

Tổng Quan

Vấn Đề Cốt Lõi

Code do AI tạo có thể **trông đúng nhưng logic sai** — thiếu security, không nhất quán giữa các module. Để làm chủ AI thay vì bị AI dẫn dắt, bạn cần một tư duy hoàn toàn khác.

Bạn Sẽ Nắm Được

- Chuyển hóa từ **Code Writer** → **Outcome Engineer**
- T-Shape model trong kỷ nguyên AI
- Context Engineering — quản lý AI như infrastructure
- Debug & kiểm chứng code do AI sinh ra
- Bảo mật và trách nhiệm đạo đức

"AI không thay thế developer — nó thay thế developer **không biết dùng AI**. Và developer biết dùng AI mà thiếu nền tảng kỹ thuật thì như nhạc trưởng không biết đọc nhạc."

Nội Dung Chương 2

Mục Lục

01

2.1

Code Writer → Outcome Engineer

03

2.3

Cognitive Load & Context Engineering

05

2.5

Đạo Đức & Bảo Mật: Trách Nhiệm Của Developer

02

2.2

T-Shape Model cho AI-Augmented Developer

04

2.4

Debugging & Verification — Kỹ Năng Sống Còn

06

Case Study

Code Writer vs. Outcome Engineer — Login Feature

Từ Code Writer sang Outcome Engineer

Khi AI đảm nhận việc viết code, giá trị của bạn dịch chuyển hoàn toàn — từ tốc độ gõ phím sang khả năng **định nghĩa vấn đề, chỉ huy AI, và kiểm chứng kết quả**. Đây là bước chuyển từ *how* sang *why & what*.



Vì Sao Tư Duy Phải Thay Đổi?

2.1.1 — Bối Cảnh

Thị trường đã chuyển dịch

- GitHub Copilot: 1,8 triệu developers dùng **mỗi ngày**
- Stack Overflow: traffic **giảm lần đầu** khi AI coding tools bùng nổ
- McKinsey: **30–40%** coding tasks có thể do AI thực hiện
- "AI-native" teams xuất hiện — **không có junior coder**

Những gì AI không thể thay thế

- **Business context** — hiểu tại sao feature này cần tồn tại
- **Trade-off decisions** — performance vs. maintainability
- **Domain knowledge** — kế toán, y tế, pháp lý
- **Trách nhiệm** — khi hệ thống lỗi, AI không bị sa thải

Câu hỏi không còn là "AI có thay thế developer không?" — mà là "Developer nào sẽ bị thay thế, và developer nào sẽ trở nên không thể thiếu?"

Intent → Specification → Verification

2.1.2 — 3 Kỹ Năng Cốt Lõi

① Xác Định Intent

Làm rõ **Tại sao** và **Cái gì** trước khi nghĩ đến *Như thế nào*.

Dùng framework Why-What-How. Không hiểu intent → không viết được spec tốt.

② Viết Specification

Spec đủ chi tiết để AI implement **không cần hỏi lại**: acceptance criteria có thể test, constraints rõ ràng, edge cases được xử lý.

Spec rõ → AI output đúng. Spec mơ hồ → AI hallucinate.

③ Kiểm Chứng Kết Quả

Xác minh **3 tầng**: Tests pass → Logic chính xác → Architecture phù hợp. Chạy được chưa đủ — phải đúng.

Tránh automation bias: đừng tin code AI chỉ vì không có lỗi.

Framework: Why — What — How

2.1.2 — Intent Definition

WHY — Mục Đích

- Business problem cần giải quyết
- Người dùng là ai?
- Hậu quả nếu không làm?
- Tiêu chí thành công là gì?

Ví dụ: "User quên mật khẩu → mất access
→ mất doanh thu"

WHAT — Yêu Cầu

- Acceptance Criteria rõ ràng, có thể test
- Non-functional requirements
- Constraints và giới hạn
- Edge cases và error states

Ví dụ: "AC-1: Link reset hết hạn sau 15 phút"

HOW — Chỉ Định Kỹ Thuật

- Tech stack và patterns bắt buộc
- Files cần tạo hoặc chỉnh sửa
- API contracts
- Performance targets cụ thể

Ví dụ: "nodemailer, bcrypt token, Redis TTL"

  **Quy tắc vàng:** Nếu bạn chưa điền đủ Why và What, bạn chưa sẵn sàng để code — dù tự viết hay dùng AI.

Kiểm Chứng 3 Tầng

2.1.2 — Outcome Verification

Tầng 1 — Tests Pass

Unit, integration, và E2E tests đều pass. Đây là điều kiện **cần**, không phải đủ — tests chất lượng kém vẫn có thể pass trong khi logic sai.

Tầng 2 — Logic Đúng

Walk through code với realistic data. Đặt câu hỏi: "Input null thì sao? Concurrent access? Timezone khác?" Tầng này đòi hỏi **domain knowledge** của bạn — AI không thể thay thế.

Tầng 3 — Architecture Đúng

Code có theo đúng patterns không? Service layer tách biệt, error handling nhất quán, không N+1 queries, không hardcode secrets. Tầng này đòi hỏi **technical depth** thực sự.

Code Writer vs. Outcome Engineer

Bảng 2.2

Khía cạnh	Code Writer (Cũ)	Outcome Engineer (Mới)
Trọng tâm	Cú pháp & ngôn ngữ	Logic & kiến trúc hệ thống
Câu hỏi đầu tiên	"Viết thế nào?"	"Tại sao cần feature này?"
Công cụ chính	IDE & Compiler	AI Agents & Context Management
Giá trị cốt lõi	Code nhanh, ít lỗi	Thiết kế giải pháp & Kiểm chứng
Xử lý lỗi	Debug thủ công từng dòng	Verify logic & Dẫn hướng AI fix
Kỹ năng cao nhất	Thuộc API & patterns	Viết spec & Đánh giá trade-offs
Trách nhiệm	Code mình viết	Mọi code — kể cả do AI tạo
Đo lường	Lines of code / ngày	Features shipped & quality



Thực Hành Why-What-How

2.1.3 — Bài Tập

Chọn 1 tình huống

- Feature **đặt lịch khám bệnh** — phòng khám 50 bác sĩ
- Feature **tìm kiếm sách** — thư viện 20.000 đầu sách tiếng Việt
- Feature **mượn/trả sách** — có phí phạt trễ hạn

Viết trong 15 phút

- **Why:** Business context (3–5 câu)
- **What:** 5 Acceptance Criteria có thể test
- **How:** Tech stack + pattern + 2 constraints
- 2 edge cases còn mơ hồ

Tiêu chí Spec tốt

- ✓ AC đủ rõ để viết automated test?
- ✓ Constraints có số liệu và đơn vị cụ thể?
- ✓ Edge cases có tính thực tế?
- ✓ Có mục "Out of Scope" để giới hạn phạm vi?



SECTION 2.2

T-Shape Model trong Kỷ Nguyên AI

T-Shape không mới — nhưng hình dạng chữ T đã thay đổi.

Chiều ngang mở rộng: AI là công cụ xuyên suốt toàn bộ SDLC, không còn là tùy chọn.

Chiều dọc sâu hơn: fundamentals trở nên sống còn — AI khuếch đại cả kiến thức lẫn lỗ hổng tư duy của bạn.

AI Across SDLC — Planning đến Deploy

2.2.1 — Chiều Ngang

SDLC Phase	AI Role	Bạn Làm Gì	Kết Quả
Planning	Business Analyst	Cung cấp context, validate output	Requirements rõ, risks xác định
Design	Architecture Consultant	Đánh giá trade-offs, chọn approach	Architecture document
Coding	Implementation Agent	Review code, enforce patterns	Code đúng spec
Testing	QA Engineer	Viết test cases, review coverage	Test suite đầy đủ
Documentation	Technical Writer	Review accuracy, update spec	Docs đồng bộ code
Deploy	DevOps Assistant	Review configs, approve pipeline	CI/CD chạy đúng

📌💡 **Critical Thinking là bắt buộc:** Ở mỗi phase, hãy hỏi: "AI có nắm đúng context không?" — "Output có hợp lý không?" — "Có gì bị bỏ sót không?"

Fundamentals — Nền Tảng Không Thể Bỏ Qua

2.2.2 — Chiều Dọc

DSA — Nhận Ra Vấn Đề Trước Khi Nó Xảy Ra

AI không tự thêm index. AI tạo N+1 queries. AI dùng array khi nên dùng Set. **Bạn cần biết để phát hiện** — không phải để tự implement từ đầu.

Design Patterns — Ra Lệnh, Không Hy Vọng

AI thường pha trộn patterns hoặc bỏ qua hoàn toàn. Thiếu chỉ định rõ trong spec, mỗi phiên agent sẽ cho một kiến trúc khác nhau. **Bạn phải định hướng** — AI không tự chọn đúng.

System Design — Không Hỏi Thì Không Có

Scalability, caching, message queues, database indexing, load balancing — AI không tự thêm trừ khi được yêu cầu. **Vòng luẩn quẩn**: không biết → không yêu cầu → AI không làm.

Domain Knowledge — Lợi Thế Bền Vững Nhất

AI biết viết code xử lý thanh toán, nhưng **không hiểu quy tắc kế toán kép**. AI đọc được file FASTQ, nhưng không biết Phred score nghĩa là gì. Đây là lợi thế mà AI không thể thay thế.

6 Design Patterns Quan Trọng Nhất

2.2.2 — Patterns Bạn Phải Nhớ

Pattern	Khi nào dùng	Ví dụ trong spec
Repository	Tách data access khỏi business logic	"Mọi database query phải qua Repository layer"
Service Layer	Tập trung toàn bộ business logic	"Mọi business logic nằm trong /services"
Factory	Tạo objects phức tạp, nhiều biến thể	"Dùng Factory khi tạo Order với nhiều loại"
Observer	Giao tiếp theo event-driven	"Dùng event emitter cho cross-module comm"
Strategy	Hoán đổi algorithm linh hoạt	"Pricing calculator dùng Strategy pattern"
Middleware	Xử lý cross-cutting concerns	"Auth, logging, error handling dùng middleware"

Không chỉ định pattern trong spec, AI sẽ tự ứng biến — Module A dùng Repository, Module B truy vấn DB trực tiếp trong controller. Kết quả: **một codebase, nhiều kiến trúc mâu thuẫn.**

Ma Trận Tự Đánh Giá Kỹ Năng

Bảng 2.4 — 2.2.3

Thang điểm: **1** = Chưa biết | **2** = Nhận biết | **3** = Cơ bản | **4** = Thành thạo | **5** = Chuyên gia

Kỹ năng	A	B	C	D	E
AI TOOLING					
Prompt Engineering	/5	/5	/5	/5	/5
AI Coding Agent (Claude Code / Cursor)	/5	/5	/5	/5	/5
Viết Spec / User Story / Intent	/5	/5	/5	/5	/5
Review & đánh giá code AI	/5	/5	/5	/5	/5
AGENTS.md / CLAUDE.md / .cursorrules	/5	/5	/5	/5	/5
Git + CI/CD	/5	/5	/5	/5	/5
FUNDAMENTALS					
Cấu trúc dữ liệu & Thuật toán	/5	/5	/5	/5	/5
Design Patterns	/5	/5	/5	/5	/5
Database Design & SQL	/5	/5	/5	/5	/5
Security Fundamentals	/5	/5	/5	/5	/5
Domain Knowledge (dự án)	/5	/5	/5	/5	/5

Phase 1: Planning — AI Như Business Analyst

2.2.4 — Thực Hành SDLC

Prompt 1 — Brainstorm Requirements

"Tôi đang xây dựng hệ thống quản lý thư viện trực tuyến cho trường đại học 5.000 sinh viên. Hãy cung cấp:

1. 15 functional requirements (chia theo actor: SV, thủ thư, admin)
2. 10 non-functional requirements
3. 5 rủi ro kỹ thuật hàng đầu kèm mitigation strategy
4. 3 milestones gợi ý cho lộ trình 20 tuần"

Prompt 2 — Phê Bình Spec

"Hãy đóng vai Senior Developer và phê bình spec Mượn Sách của tôi:

- Chỉ ra 10 điểm mơ hồ hoặc thiếu chi tiết
- Liệt kê 5 edge cases bị bỏ sót
- Đánh giá Acceptance Criteria: có testable không?
- Đề xuất cải thiện cụ thể. [Dán spec vào đây]"

❏ **Mẹo hiệu quả:** Giao AI vai reviewer thay vì creator — cách tốt nhất để phá vỡ author blindness.



Phase 2: Design — Phase 3: Coding

2.2.4 — Thực Hành SDLC

Prompt 3 — So Sánh Kiến Trúc Tìm Kiếm

"Dự án thư viện: React + Node.js + PostgreSQL. So sánh 3 phương án tìm kiếm:

A) PostgreSQL FTS (tsvector + GIN index)

B) PostgreSQL LIKE + unaccent extension

C) Elasticsearch độc lập Bối cảnh: 20K sách, 5K users, tiếng Việt, đồ án 20 tuần, không có ops team. → Phân tích: performance, hỗ trợ tiếng Việt, độ phức tạp, chi phí — và đưa ra recommendation cụ thể."

Prompt 4 — Sinh Code Từ Spec

"Implement tính năng Mượn Sách theo spec.

Constraints:

- Pattern: Controller → Service → Repository
- Error handling: throw custom AppError
- Validation: zod schema
- Naming: camelCase (files: kebab-case)
- Mỗi function $\leq$ 30 dòng

Trước khi code, liệt kê toàn bộ files sẽ tạo/sửa và chờ xác nhận."

Plan-first approach → code nhất quán, dễ review hơn

Phase 4: Testing — Phase 5: Docs

2.2.4 — Thực Hành SDLC

Prompt 5 — Viết Test Suite

"Viết test suite cho BorrowService.borrowBook() với 5 loại cases:

1. Happy path — mượn thành công
2. Validation errors — sách không tồn tại
3. Business rule violations — vượt giới hạn 5 cuốn
4. Edge cases — concurrent borrow
5. Error handling — database timeout

Stack: Jest + supertest. Mock toàn bộ database."

Đánh giá chất lượng: Dùng mutation testing — sửa logic, kiểm tra tests có FAIL không.

Prompt 6 — Tạo API Docs (OpenAPI 3.0)

"Từ BorrowController, generate API docs OpenAPI 3.0 gồm:

- Endpoints: HTTP method + path
- Request body schema & validation rules
- Response schema: success & error
- Authentication requirements
- Rate limiting
- Request/response examples"

📄 💡 **Coverage ≠ Quality:** 80% coverage với tests có nghĩa tốt hơn 100% coverage với tests vô nghĩa.

Tại Sao DSA Càng Quan Trọng Hơn Khi Có AI?

2.2.5 — DSA & Performance

"Nếu AI viết code được, tại sao phải học DSA?" — Vì DSA không để *viết* code. DSA để **đánh giá code AI tạo ra có thực sự hiệu quả không.**"

N+1

Queries

lỗi phổ biến nhất AI mắc phải

$O(n^2)$

→ $O(n)$

chỉ cần dùng Hash Map đúng chỗ

5000x

chênh lệch tốc độ

với 10K records

Bạn **không cần** thuộc lòng cách implement Red-Black Tree hay Dijkstra.

Bạn **cần biết** khi nào dùng hash map thay vì nested loop — khi nào cần index, khi nào query N+1 xảy ra, khi nào cần pagination.

Nested Loop vs Hash Map — N+1 Queries

2.2.5 — Ví Dụ 1 & 2

Ví dụ 1: Tối ưu từ $O(n^2)$ xuống $O(n)$

```
// AI tạo —  $O(n^2)$ 
for (const book1 of list1) {
  for (const book2 of list2) {
    if (book1.isbn === book2.isbn)
      duplicates.push(book1);
  }
}
// Tối ưu —  $O(n+m)$  — dùng Set
const isbnSet = new Set(
  list2.map(b => b.isbn)
);
return list1.filter(
  b => isbnSet.has(b.isbn)
);
// 10K sách: nhanh hơn 5000x
```

Ví dụ 2: Loại bỏ N+1 Queries

```
// AI tạo — 101 queries!
const books = await db.query(
  'SELECT * FROM books' // 1
);
for (const book of books) {
  book.author = await db.query(
    'SELECT * FROM authors ...'
  ); // N queries
}
// Tối ưu — 1 query với JOIN
SELECT b.*, a.name
FROM books b
JOIN authors a
ON b.author_id = a.id
```

Code AI tạo ra **trông đúng, chạy đúng** — nhưng sẽ sụp đổ khi gặp dữ liệu thực tế.

Index — Phân Trang — Cấu Trúc Dữ Liệu

2.2.5 — Ví Dụ 3, 4 & 5

Ví dụ 3: Thiếu Index

```
-- Không có index
SELECT * FROM borrow_records
WHERE user_id = 123
ORDER BY borrowed_at DESC;
-- Full scan: 100-500ms

-- Thêm index
CREATE INDEX idx_borrow
ON borrow_records
(user_id, borrowed_at DESC);
-- B-tree: 1-5ms
```

Ví dụ 4: Tải Không Giới Hạn

```
// AI: load toàn bộ
const books = await db.query(
  'SELECT * FROM books'
);
// 20K records = ~10MB JSON!

// Tối ưu: cursor pagination
const { cursor, limit=20 } = req;
const books = await db.query(
  `SELECT * FROM books
   WHERE id > $1 LIMIT $2`,
  [cursor || 0, limit]
);
```

Ví dụ 5: Array vs Set

```
// AI dùng array
const cats = [];
for (const b of books) {
  if (!cats.includes(b.cat)) {
    cats.push(b.cat); // O(n)!
  }
} // Tổng: O(n²)

// Đúng: dùng Set
return [...new Set(
  books.map(b => b.category)
)]; // O(n)
```

Pattern 1-2: Repository + Service Layer

2.2.6 — Repository & Service Patterns

Pattern 1: Repository — Tách DB khỏi Controller

```
// FAT CONTROLLER (sai)
app.get('/api/users/:id',
  async (req, res) => {
    const user = await db.query(
      // DB trong ctrl
      'SELECT * FROM users WHERE id=$1',
      [req.params.id]
    );
    ...
  });

// Đúng pattern
app.get('/api/users/:id',
  async (req, res) => {
    const user = await userService
      // ✓
      .getUserWithStats(req.params.id);
    res.json(user);
  });
```

Pattern 2: Service Layer — Phân tách trách nhiệm rõ ràng

```
// Quy tắc phân biệt:
// if/else về business rule → Service
// DB query → Repository
// Parse request/format res → Controller

async function borrowBook(userId, bookId) {
  const user = await userRepo
    .findById(userId);
  if (!user || user.status !== 'active')
    throw new AppError(403, 'Not eligible');

  const borrows = await borrowRepo
    .countActive(userId);
  if (borrows >= 5)
    throw new AppError(400, 'Max limit');
  ...
}
```



Controller

Service

Repository

Database



SECTION 2.3

Cognitive Load & Context Engineering

Làm việc với AI đòi hỏi quản lý song song hai tài nguyên hữu hạn: **context window của AI** và **cognitive load của bạn**. Cả hai đều dễ bị quá tải — hãy quản lý chúng như infrastructure, không phải để mặc kệ.

Context Window — RAM của AI (2026)

2.3.1 — Context Window

Model	Context Window	Ước tính trang sách	Ghi chú
Claude Opus 4.6	200K tokens	~300 trang	Dùng trong Claude Code
Claude Sonnet 4.6	200K tokens	~300 trang	Nhanh hơn, rẻ hơn Opus
GPT-4o	128K tokens	~190 trang	Dùng trong Copilot
Gemini 2.5 Pro	1M tokens	~1,500 trang	Dùng trong Kiro
Codex CLI (OpenAI)	128K tokens	~190 trang	Open-source

Thực tế một session Claude Code:

- System prompt + tools: ~10K tokens
- AGENTS.md + CLAUDE.md: ~3K tokens
- 10 code files agent đọc: ~30K tokens
- 20 messages lịch sử chat: ~40K tokens
- Agent reasoning (thinking): ~30K tokens
- Code agent tạo ra: ~20K tokens

Tổng: ~133K / 200K = 66% đã tiêu — chỉ còn 67K tokens để làm việc tiếp theo.

Khi Context Gắn Đầy — AI Bắt Đầu Sai

2.3.1 — Triệu Chứng

① Quên hướng dẫn ban đầu

Instructions đầu phiên bị đẩy ra khỏi vùng chú ý khi context tích lũy quá dài.

② Nhầm lẫn giữa các file

Dễ xảy ra với các file có tên hoặc cấu trúc tương tự — AI trộn lẫn nội dung.

③ Code thiếu nhất quán

Code sinh ra sau mâu thuẫn với code trước trong cùng phiên làm việc.

④ Hallucinate API / function

Bịa ra API, function hoặc library không tồn tại — xảy ra nhiều hơn khi context bị nhiễu.

⑤ Vòng lặp sửa — hòng không thoát

Agent sửa xong lại hòng, hòng xong lại sửa — lặp vô tận. **Dấu hiệu rõ nhất cần bắt đầu phiên mới ngay.**

4 Nguyên Tắc Cốt Lõi

2.3.2 — Context Engineering

① Selection — Chọn Đúng

Không dump toàn bộ codebase. Chỉ cung cấp files liên quan trực tiếp.

- ✓ CẦN: Service file, interface, caller, test, error log
- × KHÔNG: Files không liên quan, lịch sử chat cũ

② Structure — Cấu Trúc Rõ Ràng

Thông tin có cấu trúc hiệu quả hơn thông tin rời rạc. Thay vì paste nhiều đoạn chat, tổng hợp thành một file ngắn gọn.

AGENTS.md và **CLAUDE.md** là công cụ cấu trúc context tiêu chuẩn.

③ Clean — Loại Bỏ Nhiều

Xóa thông tin lỗi thời, dead code, comments thừa.

- Mỗi task → một conversation mới
- Phiên vượt 30 messages → bắt đầu lại ngay

④ Layer — Phân Tầng Context

→ **Tầng 1: Foundation** — AGENTS.md, CLAUDE.md — luôn có mặt

→ **Tầng 2: Feature** — Spec của feature đang thực hiện

→ **Tầng 3: Task** — Code, errors, tests cụ thể

Context Engineering vs. Quản Lý RAM

Bảng 2.6 — So Sánh Chiến Lược

Khía cạnh	Quản lý RAM (máy tính)	Context Engineering (AI)
Tài nguyên	RAM (GB)	Context window (tokens)
Giới hạn	Hết RAM → swap / crash	Hết context → hallucinate / quên
Chiến lược	Load on demand, cache, GC	Chọn lọc · cấu trúc · làm sạch · phân tầng
Tối ưu hóa	Memory profiling, heap analysis	Token counting, fresh sessions
Anti-pattern	Memory leak, dangling pointers	Context pollution — dump mọi thứ vào prompt
Công cụ	Task Manager, htop, Valgrind	AGENTS.md, CLAUDE.md, memory.md
Mục tiêu	Chương trình chạy ổn định	AI output chính xác và nhất quán

5 Kỹ Thuật Giảm Tải Nhận Thức

2.3.3 — Cognitive Load

1 Modularize Ruthlessly

Chia nhỏ thành từng module độc lập — mỗi module xử lý trong **một phiên agent riêng**. Giới hạn: **300–500 dòng code** mới mỗi phiên.

2 Start Fresh, Stay Focused

Mỗi task = một phiên mới. Phiên vượt **30 messages** → mở phiên mới với summary ngắn. Phiên sạch giúp cả AI lẫn bạn tập trung tối đa.

3 Plan Before You Code

Yêu cầu AI lên kế hoạch trước: files cần tạo/sửa, thứ tự thực hiện, dependencies. **Chờ approve plan** — không cho phép bắt đầu code ngay.

4 Externalize with Checklists

Working memory chỉ giữ được **4±2 items** (Miller, 1956). Checklist ngoại hóa trí nhớ — đảm bảo không bước nào bị bỏ sót.

5 Time-Box Reviews (Pomodoro)

Review tối đa **25 phút**, nghỉ 5 phút. Não bộ đạt hiệu quả cao nhất trong sprint đầu — đừng review 2.000 dòng code trong một phiên liên tục.

3 Tình Huống Context Engineering

2.3.4 — Thực Hành

Tình Huống 1: Khởi Động Ngày Mới

AI không lưu ký ức giữa các phiên. Mỗi sáng, "load context" như load save game:

"Hôm nay làm feature Auth. Spec: .spec/auth-spec.md. Hôm qua hoàn thành: login + rate limit. Hôm nay: refresh token + logout."

Phiên mới + summary ngắn = context sạch, tập trung ngay vào việc.

Tình Huống 2: Agent Mất Định Hướng

Dấu hiệu: lặp code cũ, sai naming, bỏ qua constraints. Áp dụng **Context Reset Protocol**:

- Dừng ngay, không tiếp tục
- Lưu lại progress hiện tại
- Mở phiên mới với summary súc tích
- Tiếp tục từ điểm dừng

Tình Huống 3: Đồng Bộ Nhóm 5 Người

Mỗi người dùng AI riêng → code không nhất quán. Giải pháp: **Shared Context Layer** dùng chung:

```
project-root/  
├── .spec/  
│   ├── constitution.md ← bất biến  
│   ├── architecture.md  
│   └── api-contracts.md  
├── AGENTS.md ← mọi agent đọc  
└── CLAUDE.md
```

AGENTS.md — Template Cho Đồ Án

2.3.6 — Template

```
# AGENTS.md
## Project Overview
Hệ thống quản lý thư viện XYZ.
5000 sinh viên, 20000 đầu sách.

## Tech Stack
- Frontend: React 18 + TypeScript
- Backend: Node.js 20 + Express
- Database: PostgreSQL 16 + Prisma
- Testing: Jest + Supertest

## Architecture Pattern
Controller → Service → Repository
KHÔNG đặt DB queries ngoài *.repository.ts

## Coding Conventions
- camelCase cho functions/variables
- PascalCase cho types/classes
- kebab-case cho file names
- AppError cho mọi errors
- Mọi function < 30 dòng

## Security Rules
- NEVER hardcode secrets
- NEVER paste .env vào prompt
- Parameterized queries BẮT BUỘC
- zod cho mọi user input

## Git Workflow
- Branch: feature/XX-description
- PR phải có 1 reviewer approve
```

File này là **"nguồn sự thật"** duy nhất của cả team. Khi convention thay đổi — cập nhật AGENTS.md trước, commit, rồi thông báo nhóm.

3 Team Ceremonies — Đồng Bộ Nhóm Với AI

2.3.8 — Làm Việc Nhóm

Thứ Hai — Spec Review (30 phút)

Review toàn bộ specs cho tasks trong tuần. Đảm bảo mọi người nắm rõ **ai làm gì và contract giữa các modules**. Chủ động loại bỏ xung đột trước khi merge.

Thứ Tư — Pair Programming (60 phút)

2 người cùng làm việc trong 1 phiên AI: Người 1 viết prompt, Người 2 kiểm tra logic. **Rotation hàng tuần** — mọi người hiểu code của nhau, không ai trở thành điểm nghẽn.

Thứ Sáu — Demo + Retro (45 phút)

Demo kết quả tuần. Retro nhanh: cái gì hiệu quả, cái gì cần điều chỉnh. **Cập nhật AGENTS.md** và team constitution ngay trong buổi nếu cần.

AI giúp mỗi cá nhân tạo code nhanh hơn và nhiều hơn — nhưng 5 người chạy theo 5 hướng là thảm họa. Ceremonies hoạt động như **mutex trong teamwork**: ngăn race conditions trước khi chúng xảy ra.



SECTION 2.4

Debugging & Verification

Khi AI tạo phần lớn code, công việc của bạn chuyển từ **viết sang đọc và kiểm tra** — như review code của một đồng nghiệp năng suất cao nhưng đôi khi cầu thả. Đây là kỹ năng ít được dạy, nhưng ngày càng quan trọng.

4 Đặc Trưng Code AI Cần Nhận Biết

2.4.1 — Đọc Code AI

① Trông chuyên nghiệp, nhưng logic sai

AI viết code sạch, đặt tên biến tốt, cấu trúc rõ ràng — dễ tạo **automation bias**: tin tưởng vì trông hay, không vì đúng. Đừng để hình thức đánh lừa.

② Nhất quán trong file, xung đột giữa files

AI không có memory xuyên phiên: Module A dùng camelCase, Module B dùng snake_case; error handling mỗi service một kiểu. Đây là lý do cần **Constitution + AGENTS.md**.

③ Bỏ sót edge cases

AI xử lý happy path tốt, nhưng thường bỏ qua:

- Null / undefined / empty string
- Concurrent access
- Timezone (UTC vs local)
- Unicode (tiếng Việt, emoji)
- Network failure, timeout

④ Dependencies thừa

AI có xu hướng import nhiều hơn cần thiết. Mỗi dependency dư = 1 rủi ro bảo mật + 1 điểm hồng tiềm tàng. Luôn kiểm tra: **package nào thực sự được dùng?**

TDD + AI = Combo Mạnh Nhất

2.4.3 — TDD + AI

Red: Viết tests

Green: AI viết code

Lặp lại: Thêm edge cases

Refactor: Bạn review



Ví dụ: Tests là hợp đồng với business rules

```
test('apply highest discount only', () => {
  // Business rule: chỉ áp mã cao nhất
  expect(calculateFinalPrice(100000, [
    { type: 'pct', value: 50 },
    { type: 'pct', value: 30 }
  ])).toBe(50000);
});

test('price cannot go negative', () => {
  expect(calculateFinalPrice(
    10000, [{type:'fixed',value:20000}]
  )).toBe(0);
});
```

Tại sao TDD + AI hiệu quả hơn

- Bạn định nghĩa "**đúng là gì**" — AI lo phần còn lại
- Tests là hàng rào cứng: code sai → fail ngay lập tức
- Không cần đọc từng dòng implementation — tests pass là đủ tin
- Edge cases được kiểm soát chủ động, không phó mặc cho AI

10 Lỗi Phổ Biến Nhất Trong Code AI

Bảng — 2.4.4

#	Lỗi	Tần suất	Mức nguy hiểm	Cách phát hiện
1	N+1 queries	Rất cao	Performance	Kiểm tra mọi loop chứa DB query
2	Missing input validation	Rất cao	Security	Kiểm tra mọi endpoint nhận dữ liệu từ user
3	Hardcoded secrets	Cao	Security	Grep: password, secret, key, token
4	Inconsistent error handling	Cao	Maintainability	So sánh cách xử lý lỗi giữa các module
5	No pagination	Cao	Performance	Kiểm tra mọi endpoint trả về danh sách
6	Race conditions	Trung bình	Data integrity	Tìm read-then-write không có lock
7	Missing index	Trung bình	Performance	EXPLAIN mọi query trên tập dữ liệu lớn
8	Zombie imports	Thấp	Code quality	Bật lint rule: no-unused-imports
9	Over-abstraction	Thấp	Maintainability	Hỏi: "Có hơn một implementation không?"
10	Stale comments	Thấp	Maintainability	Đối chiếu comment với logic code hiện tại

Quy Trình Debug Code AI — 6 Bước

2.4.5 — Debug Protocol

01

Mô tả triệu chứng chính xác

Không phải "nó không chạy" — mà là: *"POST /api/login trả về 500, body: {...}, error: TypeError: Cannot read property 'id' of null"*

03

Khoanh vùng layer lỗi

Controller? Service? Repository? DB? Gọi trực tiếp từng function với test data để cô lập nguyên nhân.

05

Xác nhận và kiểm thử fix

Hiểu **tại sao fix đó đúng** → viết test cho case gây lỗi → chạy toàn bộ test suite để đảm bảo không regression.

02

Kiểm tra spec trước tiên

Lỗi có thể nằm ở spec mơ hồ, không phải implementation. Ví dụ: spec không định nghĩa hành vi khi user là null.

04

Dùng AI để phân tích lỗi

Mở phiên MỚI (context sạch). Cung cấp đủ: endpoint, input, expected, actual, error, code. Yêu cầu AI phân tích nguyên nhân gốc rễ.

06

Cập nhật spec và tests

Bổ sung edge case vào spec. Thêm test case tương ứng. **Bug là thông tin — hãy document lại để nó không tái diễn.**

Debug Thực Tế: Overdue Fee

2.4.6 — Walkthrough

Bug: calculateOverdueFee trả số âm

```
// Code AI tạo — BUG ở đây!  
function calculateOverdueFee(  
  dueDate: string,  
  returnDate: string  
) {  
  const due = new Date(dueDate);  
  const ret = new Date(returnDate);  
  const diffMs =  
    due.getTime() - ret.getTime();  
  // ← ĐẢO NGƯỢC!  
  const diffDays = Math.ceil(  
    diffMs / (1000 * 60 * 60 * 24)  
  );  
  const fee = Math.max(0, diffDays * 5000);  
  return Math.min(fee, 500000);  
}
```

Mental Simulation & Fix

```
// Input: due=Feb28, ret=Mar01  
// due - ret = số ÂM (due < ret)  
// max(0, âm*5000) = 0 (BUG!)  
  
// Fix: đổi thứ tự trừ  
const diffMs =  
  ret.getTime() - due.getTime();  
  
// Verify:  
// Đúng hạn: ret=due → 0 ✓  
// Trễ 1 ngày: → 5000 ✓  
// Trễ 100 ngày: → 500000 (cap) ✓  
// Trả sớm: → 0 ✓
```

Bug chỉ là một phép trừ đảo ngược — nhưng kết quả sai hoàn toàn. **Bài học:** luôn test *cả hai chiều* (trả sớm và trả trễ) cho mọi hàm tính toán thời gian.



SECTION 2.5

Trách Nhiệm Đạo Đức

AI tạo code — nhưng **bạn chịu trách nhiệm**. Đây không chỉ là nguyên tắc đạo đức mà còn là thực tế pháp lý: khi ứng dụng gây thiệt hại, bạn và team đứng trước pháp luật — không phải Anthropic hay OpenAI.



3 Quy Tắc Vàng

2.5.1 — Human-in-the-Loop

1 — Chỉ commit code bạn thực sự hiểu.

"Vì AI viết thế" không phải lý do. Khi bị hỏi, bạn phải giải thích được từng dòng.

2 — Chỉ deploy tính năng bạn có thể giải thích.

Hiểu = nắm được logic, edge cases, và failure modes — không phải chỉ biết nó chạy.

3 — Chỉ dùng AI output bạn có thể verify.

Chưa biết cách kiểm tra output? Học cách kiểm tra trước — rồi mới dùng AI cho lĩnh vực đó.

5 Lỗi Hồng Bảo Mật Thường Gặp Trong Code AI

2.5.2 — Bảo Mật

① SQL Injection

```
// XẤU:  
'SELECT * FROM users  
WHERE email = '${email}'  
// ← NGUY HIỂM  
  
// TỐT — parameterized:  
db.query(  
  'SELECT * FROM users  
  WHERE email=$1',  
  [email]  
)
```

② Hardcoded Credentials

```
// XẤU:  
const JWT_SECRET = 'super-secret-123';  
  
// TỐT:  
const JWT_SECRET =  
  process.env.JWT_SECRET;  
if (!JWT_SECRET)  
  throw new Error('Not configured');
```

③ Thiếu Validation Đầu Vào

```
// XẤU — không validate:  
const { email, name } = req.body;  
db.createUser({ email, name });  
  
// TỐT:  
if (!isValidEmail(email))  
  return res.status(400)...;  
const clean = sanitize(name);
```

④ Dependency Lỗi Thời + ⑤ CORS Quá Rộng

```
// Sau mỗi lần AI thêm package:  
npm audit  
// Kiểm tra: package có maintained không?  
  
// XẤU — CORS open:  
app.use(cors({ origin: '*' }));  
  
// TỐT — chỉ định domain:  
app.use(cors({  
  origin: ['https://myapp.com']  
}));
```

Các nghiên cứu độc lập cho thấy **9.8% – 42.1%** code do AI tạo ra chứa lỗi hồng bảo mật — luôn kiểm tra trước khi deploy.

Security Checklist — 15 Mục Cho Mỗi PR

2.5.4 — Security Checklist

#	Kiểm tra	Mức độ
1	Không hardcoded secrets — grep: password, secret, key, token	Bắt buộc
2	Mọi DB query dùng parameterized — không nối chuỗi	Bắt buộc
3	Mọi user input được validate và sanitize	Bắt buộc
4	CORS chỉ cho phép domain cụ thể — không dùng *	Bắt buộc
5	JWT secret lấy từ env — không hardcode	Bắt buộc
6	Hash password bằng bcrypt/argon2 — không dùng MD5/SHA1	Bắt buộc
7	.gitignore bao gồm: .env, *.key, *.pem, node_modules	Bắt buộc
8	npm audit / pip audit — không có critical vulnerability	Cao
9	Rate limiting cho endpoints nhạy cảm: login, register	Cao
10	Error message không lộ thông tin nội bộ: stack trace, DB schema	Cao
11	Bắt buộc HTTPS — redirect HTTP → HTTPS	Cao
12	Không console.log dữ liệu nhạy cảm	Trung bình
13	File upload: validate type và size	Trung bình
14	Session/token có thời hạn hết hạn hợp lý	Trung bình
15	Cài Helmet.js hoặc tương đương để set HTTP security headers	Trung bình

4 Dạng Tech Debt từ AI — Nhận Diện & Ngăn Chặn

2.5.3 & 2.5.5 — Tech Debt

Dạng debt	Mô tả	Cách phát hiện	Cách ngăn
Inconsistent patterns	Module A dùng Repository, Module B dùng Active Record	Cross-module code review	Định nghĩa patterns trong Constitution
Over-engineering	Abstraction thừa cho use case đơn lẻ	"Interface này có thực sự cần thiết?"	Áp dụng YAGNI trong Constitution
Zombie code	Imports, functions tạo ra nhưng không dùng	Lint: no-unused-vars	Lint + auto-fix trong CI
Doc drift	Tài liệu lạc hậu, không khớp code hiện tại	Spot check mỗi sprint	Generate docs trực tiếp từ code

Chiến Lược: Quy Tắc 10%

Dành **10% thời gian mỗi sprint** cho debt cleanup — sprint 2 tuần tương đương 1 ngày. Thứ 6 hàng tuần: 2 giờ cố định để fix lint, chuẩn hóa naming, cập nhật docs, và xóa zombie code.

3 Kịch Bản Đạo Đức Thực Tế

2.5.6 — Thảo Luận

Kịch bản 1: Merge Không Review

Deadline demo ngày mai. AI tạo code, tests pass. Bạn muốn merge thẳng.

Thảo luận: Rủi ro thực sự là gì? Có thể giảm scope không? Có thể xin thêm thời gian không?

Kịch bản 2: Commit Code Không Hiểu

AI tạo algorithm phức tạp cho recommendation. Chạy đúng — nhưng bạn không hiểu tại sao.

Quy tắc vàng: Nếu không thể giải thích code cho nhóm, đừng commit.

Kịch bản 3: Lộ Credentials

Thành viên A vô tình paste cả file `.env` — có DB password và API keys — vào Claude.

Xử lý NGAY:

1. Rotate toàn bộ credentials đã lộ
2. Kiểm tra truy cập trái phép
3. Bổ sung rule vào `AGENTS.md`
4. Chuyển sang `.env.example` với giá trị giả

CASE STUDY

Code Writer vs. Outcome Engineer

Tính năng **Login** — React + Node.js + PostgreSQL. Hai developer, cùng một yêu cầu, hai cách tiếp cận. Kết quả: một người mất 3 ngày fix bugs, người kia xong trong 40 phút.



Code Writer: 5 Phút, 8 Lỗi Hồng

Case Study — Cách 1: Viết Code Không Suy Nghĩ

Prompt sử dụng

"Tạo tính năng login với React frontend, Node.js backend, PostgreSQL."

Code AI sinh ra (5 phút)

```
router.post('/login', async (req, res) => {  
  const { email, password } = req.body;  
  const user = await User.findByEmail(email);  
  if (!user) return res.status(401)...;  
  const match = await bcrypt.compare(...);  
  if (!match) return res.status(401)...;  
  const token = jwt.sign(  
    { id: user.id },  
    'secret-key',  
    { expiresIn: '7d' }  
  );  
  res.json({ token });  
});
```

8 Lỗi Hồng Bị Bỏ Qua

- × Không rate limiting → dễ bị brute force
- × JWT secret hardcoded → lộ trong source code
- × Token hết hạn sau 7 ngày → token bị đánh cắp dùng cả tuần
- × Không log failed attempts → tấn công không bị phát hiện
- × Không validate input → tiềm ẩn SQL injection
- × Không có refresh token → user bị logout đột ngột
- × Không có password policy → chấp nhận password "1"
- × Không có account lockout → brute force không giới hạn

Kết quả: 5 phút viết code + **3+ ngày fix bugs** + rủi ro bảo mật không kiểm soát được.

Outcome Engineer: Spec 30 Phút

Case Study — Cách 2

Feature Spec: Authentication System

Business Context

User đăng nhập để truy cập dữ liệu cá nhân.

Hệ thống bảo vệ tài khoản khỏi unauthorized access.

Target: 1.000 users, 100 concurrent logins.

Acceptance Criteria

AC-1: Đăng nhập bằng email/password

AC-2: Rate limiting: tối đa 5 lần/phút/IP

AC-3: Khóa tài khoản sau 10 lần sai — tự mở khóa sau 30 phút

AC-4: JWT access token: hết hạn sau 15 phút

AC-5: Refresh token: 7 ngày, rotate on use, thu hồi khi logout

AC-6: Password: tối thiểu 8 ký tự, gồm 1 hoa, 1 số, 1 ký tự đặc biệt

AC-7: bcrypt cost factor: 12

AC-8: Ghi log mọi lần đăng nhập (thành công/thất bại) kèm IP và timestamp

Technical Constraints

- Parameterized queries only — không nối chuỗi SQL
- Secrets từ environment variables, KHÔNG hardcoded
- Validate và sanitize toàn bộ input
- CORS: chỉ frontend domain | HTTPS bắt buộc

Edge Cases

- Đăng nhập từ các timezone khác nhau
- Đăng nhập đồng thời từ nhiều thiết bị
- Refresh token khi access token hết hạn giữa request
- Case sensitivity của email (User@blue.com vs user@blue.com)

Out of Scope (v2)

- OAuth/SSO | 2FA | Password reset qua email

Outcome Engineer: Code Hoàn Chính Trong 10 Phút

Case Study — Cách 2 (tiếp theo)

```
export async function login(
  input: unknown,
  ip: string
){
  // Validate input (AC: validate ALL input)
  const { email, password } =
    loginSchema.parse(input);

  // Rate limiting (AC-2: max 5 attempts/min/IP)
  await rateLimiter.checkLimit(
    `login:${ip}`, 5, 60
  );

  // Find user — email case-insensitive (Edge Case)
  const user = await authRepository.findByEmail(
    email.toLowerCase().trim()
  );
  if (!user) {
    logger.warn('Login failed: user not found',
      { email, ip });
    throw new AppError(401,
      'Invalid email or password');
  }

  // Check account lock (AC-3)
  if (user.lockedUntil &&
    user.lockedUntil > new Date()) {
    logger.warn('Login on locked account',
      { userId: user.id });
    throw new AppError(423,
      'Account locked. Try again later.');
```

```
  }

  // Verify password (AC-7: bcrypt cost 12)
  const isValid = await comparePassword(
    password, user.passwordHash
  );
  if (!isValid) {
    await authRepository
      .incrementFailedAttempts(user.id);
    if (user.failedAttempts + 1 >= 10) {
      const lockUntil = new Date(
        Date.now() + 30 * 60 * 1000
      );
      await authRepository
        .lockAccount(user.id, lockUntil);
    }
    throw new AppError(401,
      'Invalid email or password');
  }

  // Generate token pair (AC-4: 15min, AC-5: 7d)
  const tokens = generateTokenPair({
    userId: user.id, role: user.role
  });
  logger.info('Login successful',
    { userId: user.id, ip });
  return {
    accessToken: tokens.accessToken,
    refreshToken: tokens.refreshToken,
    expiresIn: 900
  };
}
```

Agent triển khai đầy đủ AC-2, AC-3, AC-4, AC-5, AC-7, AC-8 trực tiếp trong code. Edge case email case-insensitive được xử lý. Logging nhất quán. Error message không tiết lộ sự tồn tại của tài khoản.

So Sánh Hai Cách Tiếp Cận

Bảng 2.8 — Case Study

Tiêu chí	Code Writer	Outcome Engineer
Thời gian ban đầu	5 phút	40 phút
Thời gian fix issues	3+ ngày	< 1 giờ
Tổng thời gian thực	3+ ngày	< 2 giờ
Bảo mật	8 lỗ hổng	0 lỗ hổng đã biết
Test coverage	Basic (happy path only)	Full (mọi AC + edge cases)
Tái sử dụng	Không có spec	Spec là tài liệu sống
Team hiểu code	Chỉ người tạo	Ai đọc spec đều hiểu
Đánh giá đóng góp	Khó trace	Spec = deliverable rõ ràng
Mở rộng cho v2	Rewrite từ đầu	Update spec, re-run agent

Outcome Engineer Nhanh Hơn — Tính Cả Vòng Đời

40 phút viết spec so với 3+ ngày fix bugs. Khi tính tổng thời gian, Outcome Engineer nhanh hơn gấp nhiều lần.

Spec Là Khoản Đầu Tư, Không Phải Chi Phí

30 phút viết spec tiết kiệm 3 ngày fix bugs — và là cách duy nhất để cả team cùng hiểu một tính năng.



Outcome Engineer Challenge — 80 Phút

Bài Tập Thực Chiến Tổng Hợp

Phase	Hoạt động	Người thực hiện	Thời gian
Phase 1: Spec	Phân công: 1 viết Why, 2 viết AC + constraints, 1 viết edge cases, 1 dùng AI brainstorm	Cả nhóm	20 phút
Phase 2: TDD	Viết tests cho toàn bộ AC; dùng AI phát hiện thêm 5 edge cases	Cả nhóm	15 phút
Phase 3: Implement	Nạp spec + tests cho agent → duyệt plan → chạy implementation	Agent + Nhóm	15 phút
Phase 4: Verify	Kiểm tra: tests pass, logic đúng, architecture hợp lý, security checklist sạch	Cả nhóm	20 phút
Phase 5: Retro	Spec còn thiếu gì? So sánh với Vibe Coding. Cập nhật Skill Matrix.	Cả nhóm	10 phút

Đầu ra: 1 feature hoàn chỉnh gồm **spec + tests + code + verification + retro notes**. Lưu toàn bộ artifacts làm tài liệu học tập.

Tóm Tắt Chương 2

Tổng Kết

2.1 — Outcome Engineer

Developer chuyển từ *How* sang *Why & What*. 3 kỹ năng cốt lõi: Intent Definition, Spec Writing, Outcome Verification.

2.2 — T-Shape Mở Rộng

Ngang = AI xuyên suốt SDLC + Critical Thinking. Dọc = DSA, Patterns, Domain Knowledge — quan trọng hơn bao giờ hết.

2.3 — Context Là Hạ Tầng

4 nguyên tắc: Chọn lọc, Cấu trúc, Làm sạch, Phân tầng. 5 kỹ thuật giảm Cognitive Load. AGENTS.md = shared context cho toàn team.

2.4 — TDD + AI: Kiểm Soát Chất Lượng

Đọc code AI với tư duy phản biện. TDD = hàng rào kiểm soát chất lượng. Dùng AI phát hiện edge cases. Debug theo 6 bước có hệ thống.

2.5 — Human-in-the-Loop: Không Thỏa Hiệp

Không paste secrets vào AI. Kiểm tra 5 nhóm lỗ hổng bảo mật phổ biến. 15-item security checklist mỗi PR. Quản lý tech debt chủ động với 10% Rule.

  **Case Study:** 40 phút Outcome Engineer vượt trội 3 ngày Code Writer khi tính full lifecycle. **Spec là đầu tư — không phải chi phí.**

Bạn Đã Sẵn Sàng Khi...

Checklist Hoàn Thành Chương 2

- ☐ Hiểu rõ chuyển dịch Code Writer → Outcome Engineer
- ☐ Viết được Why-What-How cho một feature
- ☐ Tự đánh giá Skill Matrix, xác định điểm cần cải thiện
- ☐ Áp dụng được 4 nguyên tắc Context Engineering
- ☐ Phân biệt được 4 đặc trưng của code do AI sinh ra
- ☐ Thực hành TDD + AI ít nhất một lần
- ☐ Nắm 15-item security checklist
- ☐ Hiểu mục đích và cách dùng AGENTS.md